簡易 Zig

JW Lee

簡易 **Zig**

Zig 程式語言嘅完整初學者指南

作者：*JW Lee*

前言

歡迎嚟到 簡易 **Zig**！呢本書係專為想由零開始學 Zig 程式語言嘅初學者而設計。無論你係由 Python、JavaScript、C 過嚟，定係第一次學程式語言，呢本書都會一步一步咁教你。

乜嘢係 **Zig**？

Zig 係一種現代系統程式語言，由 Andrew Kelley 喺 2016 年創建。佢嘅設計係做一個「更好嘅 C」——提供對記憶體同硬件嘅低階控制，同時避免好多令 C 同 C++ 難用同容易出錯嘅陷阱。

```
┌──────────────────────────────────────────────────────────┐
│                    程式語言光譜                          │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  高階                                          低階      │
│  (易用)                                       (快速)     │
│                                                          │
│  Python ─── JavaScript ─── Go ─── Zig ─── C ─── ASM      │
│     │           │           │       │       │       │    │
│  腳本語言    網頁應用    服務程式  系統程式  作業系統  硬件 │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

點解揀 **Zig**？

1. 簡單

唔似 C++，Zig 冇隱藏嘅控制流程、冇隱藏嘅記憶體分配、冇運算子重載。你睇到嘅就係你得到嘅。

2. 安全但冇垃圾回收

Zig 喺編譯時捕捉好多錯誤。佢預設冇空指標、有陣列邊界檢查、同埋顯式錯誤處理。

3. 效能

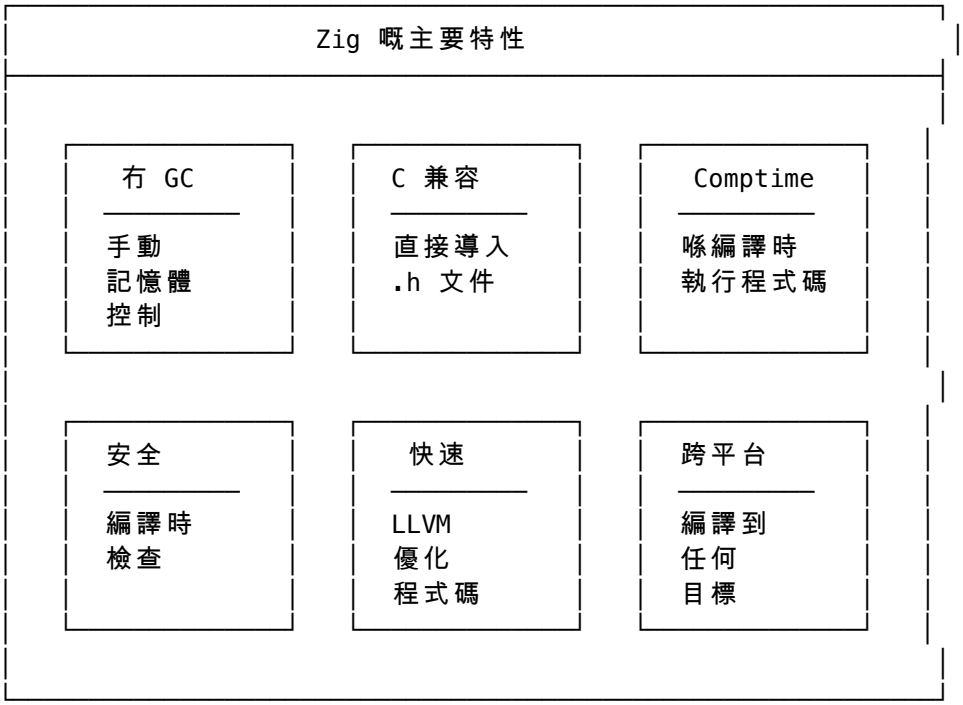
Zig 編譯成高度優化嘅機器碼，可以同 C 同 C++ 媲美。Uber 等公司都喺生產環境用緊佢。

4. C 互操作性

Zig 可以直接導入同使用 C 頭文件。唔使綁定。即係話你可以用幾十年嘅 C 程式庫。

5. Comptime（編譯時執行）

Zig 可以喺編譯時執行程式碼，令強大嘅元程式設計成為可能，而唔需要宏或模板。



設置 Zig

安裝

macOS (Homebrew):

```
brew install zig
```

Linux:

```
# 由 ziglang.org 下載並解壓
wget https://ziglang.org/download/zig-linux-x86_64.tar.xz
tar -xf zig-linux-x86_64.tar.xz
export PATH=$PATH:./zig-linux-x86_64
```

Windows: 由 https://ziglang.org/download/ 下載安裝程式

驗證安裝

```
zig version
```

你嘅第一個命令

```
# 建立一個叫 hello.zig 嘅文件，然後執行：
zig run hello.zig
```

點樣用呢本書

每一章介紹一個概念，包括：1. 解釋 - 呢個概念係乜同點解重要 2. 圖解 - 事物運作方式嘅視覺表示 3. 程式碼範例 - 你可以自己執行嘅工作程式碼 4. 輸出 - 執行程式碼時你應該睇到嘅嘢 5. 要點 - 重要收穫嘅總結

開始你嘅 Zig 之旅啦！

第一章：Hello World

每一個程式設計之旅都由 Hello World 開始。呢個簡單嘅程式教你 Zig 程式嘅基本結構。

理解程式結構

```
┌──────────────────────────────────────────────────────────┐
│                    Zig 程式結構                          │
├──────────────────────────────────────────────────────────┤
│                                                          │
│    ┌──────────────────────────────┐                      │
│    │ 1. 導入                      │                      │
│    │    const std = @import("std");│                     │
│    └──────────────────────────────┘                      │
│                    │                                     │
│                    ▼                                     │
│    ┌──────────────────────────────┐                      │
│    │ 2. 常量同全域變數            │                      │
│    │    const MAX = 100;          │                      │
│    └──────────────────────────────┘                      │
│                    │                                     │
│                    ▼                                     │
│    ┌──────────────────────────────┐                      │
│    │ 3. 函數                      │                      │
│    │    fn helper() { ... }       │                      │
│    └──────────────────────────────┘                      │
│                    │                                     │
│                    ▼                                     │
│    ┌──────────────────────────────┐                      │
│    │ 4. MAIN 函數（入口點）       │                      │
│    │    pub fn main() void { ... }│                      │
│    └──────────────────────────────┘                      │
│                                                          │
```

└──┘

你嘅第一個程式

```zig
// ============================================================
// 第一章：Hello World - 你嘅第一個 Zig 程式
// ============================================================

// 步驟 1：導入標準庫
// @import 函數帶入外部程式碼
// "std" 係 Zig 嘅標準庫，有好多有用嘅函數
const std = @import("std");

// 步驟 2：定義 main 函數
// - "pub" 表示公開（可以由外部調用）
// - "fn" 聲明一個函數
// - "main" 係入口點 - 程式由呢度開始
// - "void" 表示呢個函數冇返回值
pub fn main() void {
    // 步驟 3：輸出到控制台
    // std.debug.print() 將文字輸出到終端
    // \n 建立換行
    // .{} 係一個空嘅元組，用嚟放格式參數
    std.debug.print("Hello, World!\n", .{});
}
```

輸出：

```
Hello, World!
```

拆解每個部分

導入語句

```zig
const std = @import("std");
```

呢行做咗三件事：1. `@import("std")` - 載入 Zig 嘅標準庫 2. `const std` - 建立一個叫 `std` 嘅常量 3. `=` - 將程式庫賦值畀常量

可以咁理解：

```
┌──────────────────────────────────────────────────────────────────┐
│                        @import("std")                            │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   標準庫內容：                                                    │
│   ┌──────────────┐  ┌──────────────┐  ┌──────────────┐          │
│   │   debug      │  │    mem       │  │    fmt       │          │
│   │  .print()    │  │   .copy()    │  │  .format()   │          │
│   └──────────────┘  └──────────────┘  └──────────────┘          │
│                                                                  │
│   ┌──────────────┐  ┌──────────────┐  ┌──────────────┐          │
│   │    heap      │  │    fs        │  │    os        │          │
│   │ .allocator() │  │ .openFile()  │  │  .system()   │          │
│   └──────────────┘  └──────────────┘  └──────────────┘          │
```

```
│                                                                │
└────────────────────────────────────────────────────────────────┘
```

Main 函數

```zig
pub fn main() void {
    // 程式碼寫喺呢度
}
```

- pub - 令函數對運行時可見
- fn - 聲明函數嘅關鍵字
- main - 被認作程式入口點嘅特殊名
- () - 空括號表示冇參數
- void - 返回類型（冇返回值）
- { } - 大括號包含函數主體

Print 語句

```zig
std.debug.print("Hello, World!\n", .{});
```

我哋拆解吓：- std - 我哋導入嘅標準庫 - .debug - std 入面嘅調試工具命名空間 - .print() - 輸出格式化文字嘅函數 - "Hello, World!\n" - 格式字串（要印嘅文字）- \n - 換行嘅轉義序列 - .{} - 一個空嘅匿名結構字面量（元組），用嚟放參數

用變數印嘢

而家我哋用變數令程式更動態：

```zig
const std = @import("std");

pub fn main() void {
    // 聲明一個字串常量
    // "Zig" 會自動被識別為指向位元組嘅指標
    const name = "Zig";

    // 聲明一個整數常量
    // : u32 明確指定類型為無符號 32 位元
    const year: u32 = 2016;

    // 用格式說明符印嘢：
    // {s} – 格式化為字串
    // {d} – 格式化為十進制數字
    std.debug.print(" 歡迎嚟到 {s}!\n", .{name});
    std.debug.print("{s} 喺 {d} 年創建。\n", .{ name, year });

    // 元組入面有多個值
    const version = "0.11";
    const is_stable = true;
    std.debug.print(" 版本：{s}，穩定：{}\n", .{ version, is_stable });
}
```

輸出：

```
歡迎嚟到 Zig!
```

Zig 喺 2016 年創建。
版本：0.11，穩定：true

格式說明符參考

格式說明符	
說明符	描述
{s}	字串
{d}	十進制數字（整數）
{x}	十六進制（小寫）
{X}	十六進制（大寫）
{b}	二進制
{c}	字元
{}	預設格式（自動偵測）
{any}	印任何類型（用於調試）
{?}	可選值
{e}	科學記數法
修飾符	
{d:5}	最小寬度 5
{d:0>5}	右對齊，用零填充，寬度 5
{d:<5}	左對齊，寬度 5
{d:.2}	2 位小數（用於浮點數）

進階印嘢範例

```zig
const std = @import("std");

pub fn main() void {
    // 數字格式化
    const num: i32 = 42;
    const float: f64 = 3.14159265359;
    const byte: u8 = 255;

    std.debug.print(" 十進制：      {d}\n", .{num});
    std.debug.print(" 十六進制： 0x{x}\n", .{byte});      // 0xff
    std.debug.print(" 二進制：   0b{b}\n", .{byte});      // 0b11111111
    std.debug.print(" 浮點數：      {d:.2}\n", .{float});    // 3.14
    std.debug.print(" 科學：  {e}\n", .{float});       // 3.14159e+00

    // 填充同對齊
    std.debug.print(" 右對齊： |{d:>10}|\n", .{num}); // |        42|
    std.debug.print(" 左對齊：  |{d:<10}|\n", .{num}); // |42        |
    std.debug.print(" 零填充：   |{d:0>10}|\n", .{num});// |0000000042|
}
```

輸出：

```
十進制：      42
十六進制：  0xff
二進制：    0b11111111
浮點數：      3.14
科學：   3.14159265359e0
右對齊：  |        42|
左對齊：   |42        |
零填充：    |0000000042|
```

要點

1. 每個 **Zig** 程式都需要一個 **`main`** 函數 - 呢度係執行開始嘅地方
2. 用之前要導入 - 用 `@import` 帶入程式庫
3. 格式字串有類型檢查 - Zig 喺編譯時驗證你嘅格式說明符同參數匹配
4. 分號係必需嘅 - 唔似 JavaScript，你唔可以跳過佢哋
5. 註釋用 `//` - 一行入面 `//` 之後嘅所有嘢都會被忽略

第二章：變數

變數係儲存值嘅容器。Zig 對可以改變嘅值同唔可以改變嘅值有明確區分。

兩種變數類型

```
┌──────────────────────────────────────────────────────────┐
│                    Zig 入面嘅變數類型                     │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   ┌──────────────────┐      ┌──────────────────┐        │
│   │      const       │      │       var        │        │
│   │   ( 不可變 )     │      │    ( 可變 )      │        │
│   ├──────────────────┤      ├──────────────────┤        │
│   │                  │      │                  │        │
│   │ const x = 5;     │      │ var x = 5;       │        │
│   │ x = 10; // 錯誤!  │      │ x = 10; // 得!    │        │
│   │                  │      │                  │        │
│   │ • 唔可以改變      │      │ • 可以改變        │        │
│   │ • 首選           │      │ • 需要時先用      │        │
│   │ • 線程安全        │      │ • 要小心          │        │
│   │ • 可優化         │      │                  │        │
│   └──────────────────┘      └──────────────────┘        │
│                                                          │
│   規則：除非你需要修改，否則永遠用 const！                  │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

常量 (const)

常量係初始化之後永遠唔會改變嘅值。佢哋係 Zig 入面嘅預設選擇。

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 常量 - 永遠唔會改變嘅值
    // ========================================

    // 類型係推斷嘅（Zig 自己搞掂）
    const greeting = "Hello";           // []const u8（字串）
    const answer = 42;                   // comptime_int
    const pi = 3.14159;                  // comptime_float

    // 類型係顯式嘅（你指定）
    const age: u32 = 25;                 // 無符號 32 位元整數
    const temperature: f64 = 98.6;       // 64 位元浮點數
    const is_valid: bool = true;         // 布林值

    std.debug.print("greeting: {s}\n", .{greeting});
    std.debug.print("answer: {d}\n", .{answer});
    std.debug.print("pi: {d:.5}\n", .{pi});
    std.debug.print("age: {d}\n", .{age});
    std.debug.print("temperature: {d}\n", .{temperature});
    std.debug.print("is_valid: {}\n", .{is_valid});

    // 呢個會導致編譯錯誤：
    // age = 26;  // 錯誤：無法賦值畀常量
}
```

輸出：

```
greeting: Hello
answer: 42
pi: 3.14159
age: 25
temperature: 98.6
is_valid: true
```

變數 (var)

變數可以喺初始化之後改變。只喺需要時先用佢哋。

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 變數 - 可以改變嘅值
    // ========================================

    var counter: i32 = 0;
    std.debug.print(" 初始計數器：{d}\n", .{counter});

    // 修改變數
    counter = counter + 1;  // 而家係 1
    std.debug.print("+1 之後：{d}\n", .{counter});
```

```zig
    counter += 10;          // counter = counter + 10 嘅簡寫
    std.debug.print("+=10 之後：{d}\n", .{counter});

    counter *= 2;           // 加倍
    std.debug.print("*=2 之後：{d}\n", .{counter});

    // 用 var 聲明嘅變數必須有類型
    var name: []const u8 = "Alice";
    std.debug.print(" 名：{s}\n", .{name});

    name = "Bob";           // 改變值
    std.debug.print(" 新名：{s}\n", .{name});
}
```

輸出：

```
初始計數器：0
+1 之後：1
+=10 之後：11
*=2 之後：22
名：Alice
新名：Bob
```

變數嘅記憶體佈局

```
┌──────────────────────────────────────────────────────────────────────┐
│                              記憶體佈局                              │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   當你寫：var x: i32 = 42;                                           │
│                                                                      │
│   堆疊記憶體：                                                       │
│   ┌──────────────┬──────────┬──────────────┐                        │
│   │  地址        │  值      │  變數        │                        │
│   ├──────────────┼──────────┼──────────────┤                        │
│   │  0x1000      │   42     │     x        │ ← 4 位元組 (i32)       │
│   │  0x1004      │   ...    │  (下一個)    │                        │
│   └──────────────┴──────────┴──────────────┘                        │
│                                                                      │
│   唔同類型用唔同嘅記憶體量：                                         │
│   ┌──────────┬──────────┬──────────────────────────┐                │
│   │ 類型     │ 大小     │ 記憶體可視化             │                │
│   ├──────────┼──────────┼──────────────────────────┤                │
│   │ u8       │ 1 位元組 │ [  ]                     │                │
│   │ u16      │ 2 位元組 │ [    ]                   │                │
│   │ u32      │ 4 位元組 │ [        ]               │                │
│   │ u64      │ 8 位元組 │ [                ]       │                │
│   │ bool     │ 1 位元組 │ [  ]                     │                │
│   └──────────┴──────────┴──────────────────────────┘                │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

未定義值

有時你想聲明一個變數但之後先賦值。用 undefined：

```
const std = @import("std");

pub fn main() void {
    // ========================================
    // UNDEFINED - 而家聲明，之後賦值
    // ========================================

    // 聲明但唔初始化
    // 警告：喺賦值之前使用未定義嘅值係未定義行為！
    var result: i32 = undefined;

    // ... 一啲決定值嘅程式碼 ...

    // 而家賦值
    result = calculateSomething();

    std.debug.print(" 結果：{d}\n", .{result});
}

fn calculateSomething() i32 {
    return 42;
}
```

輸出：

結果：42

```
┌──────────────────────────────────────────────────────────┐
│                      UNDEFINED 警告                      │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   ⚠  危險：讀取未定義值係未定義行為（UB）。               │
│       程式可能：                                         │
│                                                          │
│       • 崩潰                                             │
│       • 返回垃圾值                                       │
│       • 行為不可預測                                     │
│       • 喺調試時正常但喺發布時失敗                       │
│                                                          │
│   安全模式：                                             │
│   ┌──────────────────────────────────────┐               │
│   │ var x: i32 = undefined;              │               │
│   │ x = computeValue();  // 必須先賦值！ │               │
│   │ use(x);              // 而家可以安全使用 │           │
│   └──────────────────────────────────────┘               │
│                                                          │
│   唔安全模式（唔好咁做）：                               │
│   ┌──────────────────────────────────────┐               │
│   │ var x: i32 = undefined;              │               │
│   │ use(x);  // UB！x 有垃圾值           │               │
```

```
│    └──────────────────────────────────────────────────┘              │
│                                                                       │
└───────────────────────────────────────────────────────────────────────┘
```

區塊表達式

Zig 允許你用區塊嚟計算複雜嘅初始值：

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 區塊表達式 - 複雜初始化
    // ========================================

    // 用區塊嚟計算一個值
    const result = blk: {
        var temp: i32 = 10;
        temp *= 2;           // 20
        temp += 5;           // 25
        temp = temp * temp;  // 625
        break :blk temp;     // 返回值
    };

    std.debug.print(" 結果：{d}\n", .{result});

    // 用於條件初始化
    const value: i32 = 42;
    const description = blk: {
        if (value < 0) {
            break :blk " 負數";
        } else if (value == 0) {
            break :blk " 零";
        } else {
            break :blk " 正數";
        }
    };

    std.debug.print("{d} 係 {s}\n", .{ value, description });
}
```

輸出：

```
結果：625
42 係 正數
```

編譯時常量

以 comptime 為前綴或喺頂層定義嘅常量會喺編譯時求值：

```zig
const std = @import("std");

// ========================================
// 頂層常量（編譯時）
// ========================================
```

```zig
// 呢啲係喺程式運行之前計算嘅
const SECONDS_PER_MINUTE = 60;
const MINUTES_PER_HOUR = 60;
const HOURS_PER_DAY = 24;

// 由其他常量計算（仍然係編譯時）
const SECONDS_PER_HOUR = SECONDS_PER_MINUTE * MINUTES_PER_HOUR;
const SECONDS_PER_DAY = SECONDS_PER_HOUR * HOURS_PER_DAY;

// 喺編譯時執行嘅函數
fn factorial(n: u64) u64 {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

// 預先計算嘅階乘
const FACT_5 = factorial(5);   // 喺編譯時計算！
const FACT_10 = factorial(10); // 都係編譯時！

pub fn main() void {
    std.debug.print(" 每小時秒數：{d}\n", .{SECONDS_PER_HOUR});
    std.debug.print(" 每日秒數：{d}\n", .{SECONDS_PER_DAY});
    std.debug.print("5! = {d}\n", .{FACT_5});
    std.debug.print("10! = {d}\n", .{FACT_10});
}
```

輸出：

```
每小時秒數：3600
每日秒數：86400
5! = 120
10! = 3628800
```

```
┌──────────────────────────────────────────────────────────────┐
│                 編譯時 vs 運行時                                 │
├──────────────────────────────────────────────────────────────┤
│                                                                │
│   源程式碼                                                       │
│       │                                                        │
│       ▼                                                        │
│   ┌──────────────────────────────────────────┐                 │
│   │              編譯時                        │                 │
│   │   • 頂層嘅 const                           │                 │
│   │   • comptime 區塊                          │                 │
│   │   • 類型計算                               │                 │
│   │   • 泛型實例化                             │                 │
│   │                                            │                 │
│   │   例子：const X = factorial(10);           │                 │
│   │              而家計算！唔係運行時。          │                 │
│   └──────────────────────────────────────────┘                 │
│       │                                                        │
│       ▼                                                        │
│   可執行文件（X 已經係 3628800）                                   │
```

```
│        │                                                  │
│        ▼                                                  │
│   ┌──────────────────────────────────────────┐           │
│   │              運行時                       │           │
│   │   • var 變數                              │           │
│   │   • 用戶輸入                              │           │
│   │   • 用運行時值調用嘅函數                  │           │
│   │                                           │           │
│   │   例子：var x = getUserInput();           │           │
│   │              而家計算，喺執行期間。       │           │
│   └──────────────────────────────────────────┘           │
│                                                           │
└───────────────────────────────────────────────────────────┘
```

命名慣例

```
┌──────────────────────────────────────────────────────────┐
│                      命名慣例                            │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   類型              │  慣例            │  例子           │
│   ──────────────────────────────────────────────────     │
│   局部變數          │  snake_case      │  my_variable    │
│   常量              │  snake_case 或   │  max_size       │
│                     │  SCREAMING_CASE  │  MAX_SIZE       │
│   函數              │  camelCase       │  calculateSum   │
│   類型/結構         │  PascalCase      │  MyStruct       │
│   編譯時            │  SCREAMING_CASE  │  BUFFER_SIZE    │
│                                                          │
│   例子：                                                 │
│   ┌──────────────────────────────────────────────┐       │
│   │  const BUFFER_SIZE = 1024;  // 編譯時常量     │       │
│   │  var user_count: u32 = 0;   // 可變變數       │       │
│   │  const User = struct {};    // 類型定義       │       │
│   │  fn getUserName() {}        // 函數           │       │
│   └──────────────────────────────────────────────┘       │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

要點

1. 用 **const** 多過 **var** - 不可變性防止錯誤
2. 類型可以推斷或顯式 - Zig 可能時會自己搞掂類型
3. **undefined** 係危險嘅 - 只喺你會喺讀取之前賦值時先用
4. 區塊表達式 - 用標籤區塊嚟做複雜初始化
5. 編譯時求值 - 頂層常量喺運行時之前計算

第三章：類型

Zig 係一種靜態類型語言，即係每個值都有一個喺編譯時已知嘅類型。理解類型係寫正確 Zig 程式碼嘅基礎。

類型系統概覽

```
┌──────────────────────────────────────────────────────────────────────┐
│                          Zig 類型層次                                 │
├──────────────────────────────────────────────────────────────────────┤
│                                                                       │
│                              所有類型                                 │
│                                 │                                     │
│                  ┌──────────────┴──────────────┐                      │
│                  │                             │                      │
│          原始類型                          複合類型                   │
│                  │                             │                      │
│        ┌─────────┴─────────┐         ┌─────────┴─────────┐            │
│        │                   │         │                   │            │
│      數值               其他          陣列              結構          聯合  │
│        │                   │         │                   │            │
│    ┌───┴───┐           ┌───┴───┐     切片              枚舉          可選  │
│    │       │           │       │     指標                              │
│  整數   浮點         布林     Void    向量                              │
│                                                                       │
└──────────────────────────────────────────────────────────────────────┘
```

整數類型

Zig 提供對整數大小嘅精細控制：

```
┌──────────────────────────────────────────────────────────────────────┐
│                          整數類型                                     │
├──────────────────────────────────────────────────────────────────────┤
│                                                                       │
│   有符號（可以係負數）              無符號（只係正數）                  │
│   ────────────────────────         ────────────────────────           │
│   i8   : -128 到 127               u8   : 0 到 255                    │
│   i16  : -32768 到 32767           u16  : 0 到 65535                  │
│   i32  : -21億 到 21億             u32  : 0 到 42億                   │
│   i64  : 超大範圍                  u64  : 0 到 超大                   │
│   i128 : 更大                      u128 : 更大                        │
│                                                                       │
│   特殊類型：                                                          │
│   ──────────────                                                      │
│   isize : 有符號指標大小（32 或 64 位元）                              │
│   usize : 無符號指標大小（用於陣列索引）                               │
│                                                                       │
│   任意寬度：                                                          │
│   ────────────────                                                    │
│   u1, u2, u3, ..., u65535   （任何位元寬度！）                         │
│   i1, i2, i3, ..., i65535                                             │
```

```
│                                                                │
└────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 有符號整數（可以係負數）
    // ========================================

    const tiny: i8 = -128;        // 8 位元：-128 到 127
    const small: i16 = -32768;    // 16 位元
    const medium: i32 = -2147483648; // 32 位元
    const large: i64 = -9223372036854775808; // 64 位元

    std.debug.print("i8 最小值：  {d}\n", .{tiny});
    std.debug.print("i16 最小值：{d}\n", .{small});
    std.debug.print("i32 最小值：{d}\n", .{medium});
    std.debug.print("i64 最小值：{d}\n", .{large});

    // ========================================
    // 無符號整數（只係正數）
    // ========================================

    const byte: u8 = 255;          // 8 位元：0 到 255
    const word: u16 = 65535;       // 16 位元
    const dword: u32 = 4294967295; // 32 位元
    const qword: u64 = 18446744073709551615; // 64 位元

    std.debug.print("\nu8 最大值：  {d}\n", .{byte});
    std.debug.print("u16 最大值：{d}\n", .{word});
    std.debug.print("u32 最大值：{d}\n", .{dword});
    std.debug.print("u64 最大值：{d}\n", .{qword});

    // ========================================
    // 任意位元寬度
    // ========================================

    const nibble: u4 = 15;         // 4 位元：0 到 15
    const six_bits: u6 = 63;       // 6 位元：0 到 63
    const twelve: u12 = 4095;      // 12 位元：0 到 4095

    std.debug.print("\nu4 最大值：  {d}\n", .{nibble});
    std.debug.print("u6 最大值：  {d}\n", .{six_bits});
    std.debug.print("u12 最大值：{d}\n", .{twelve});
}
```

輸出：

```
i8 最小值：  -128
i16 最小值：-32768
i32 最小值：-2147483648
i64 最小值：-9223372036854775808

u8 最大值：  255
```

```
u16 最大值：65535
u32 最大值：4294967295
u64 最大值：18446744073709551615

u4 最大值：  15
u6 最大值：  63
u12 最大值：4095
```

浮點類型

```
┌──────────────────────────────────────────────────────────────────────┐
│                              浮點類型                                 │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   類型     大小       精度          範圍（約）                        │
│   ────     ────       ──────────    ──────────────                    │
│   f16      16 位元    ~3 位數       ±65504                           │
│   f32      32 位元    ~7 位數       ±3.4 × 10^38                     │
│   f64      64 位元    ~15 位數      ±1.8 × 10^308                    │
│   f128     128 位元   ~33 位數      ±1.2 × 10^4932                   │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 浮點數
    // ========================================

    const pi_32: f32 = 3.14159265358979;  // 精度較低
    const pi_64: f64 = 3.14159265358979;  // 精度較高

    std.debug.print("f32 pi：{d:.10}\n", .{pi_32});
    std.debug.print("f64 pi：{d:.14}\n", .{pi_64});

    // 科學記數法
    const avogadro: f64 = 6.022e23;  // 6.022 × 10^23
    const planck: f64 = 6.626e-34;  // 6.626 × 10^-34

    std.debug.print(" 阿伏伽德羅：{e}\n", .{avogadro});
    std.debug.print(" 普朗克：{e}\n", .{planck});

    // 精度好重要！
    const a: f32 = 0.1;
    const b: f32 = 0.2;
    const sum: f32 = a + b;
    std.debug.print("\n0.1 + 0.2 = {d:.20}\n", .{sum});
    std.debug.print("（因為二進制表示唔係精確 0.3）\n", .{});
}
```

布林類型

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 布林 - true 或 false
    // ========================================

    const is_active: bool = true;
    const is_empty: bool = false;

    std.debug.print("is_active：{}\n", .{is_active});
    std.debug.print("is_empty：{}\n", .{is_empty});

    // 由比較產生嘅布林
    const x: i32 = 10;
    const y: i32 = 20;

    const is_equal = x == y;     // false
    const is_less = x < y;       // true
    const is_greater = x > y;    // false

    std.debug.print("\n{d} == {d}：{}\n", .{ x, y, is_equal });
    std.debug.print("{d} < {d}：{}\n", .{ x, y, is_less });
    std.debug.print("{d} > {d}：{}\n", .{ x, y, is_greater });

    // 記憶體入面嘅布林係 1 位元組
    std.debug.print("\nbool 嘅大小：{d} 位元組\n", .{@sizeOf(bool)});
}
```

類型轉換

```
┌──────────────────────────────────────────────────────────────┐
│                          類型轉換                            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   擴展（安全 - 自動）                                        │
│   ──────────────────────────                                 │
│   u8 ──────► u16 ──────► u32 ──────► u64                     │
│        得          得          得                             │
│                                                              │
│   收窄（危險 - 需要 @intCast）                               │
│   ──────────────────────────────────                         │
│   u32 ──────► u16 ──────► u8                                 │
│        ⚠          ⚠          可能丟失數據！                  │
│                                                              │
│   轉換函數：                                                 │
│   ──────────────────                                         │
│   @intCast       ：整數類型之間轉換                          │
│   @floatCast     ：浮點類型之間轉換                          │
│   @intFromFloat  ：浮點到整數（截斷）                        │
```

```
│   @floatFromInt  : 整數到浮點                                    │
│   @truncate      : 強制截斷                                      │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 擴展（安全，隱式）
    // ========================================

    const small: u8 = 42;
    const medium: u16 = small;   // u8 放得入 u16，得
    const large: u32 = medium;   // u16 放得入 u32，得

    std.debug.print(" 擴展：{d} -> {d} -> {d}\n", .{ small, medium, large });

    // ========================================
    // 收窄（危險，需要 @intCast）
    // ========================================

    const big: u32 = 200;
    const tiny: u8 = @intCast(big);  // 你必須確定佢放得入！

    std.debug.print(" 收窄：{d} -> {d}\n", .{ big, tiny });

    // 警告：如果 big > 255 會崩潰
    // const too_big: u32 = 300;
    // const crash: u8 = @intCast(too_big);  // 運行時錯誤！

    // ========================================
    // 浮點轉換
    // ========================================

    const float64: f64 = 3.14159265358979;
    const float32: f32 = @floatCast(float64);  // 丟失精度

    std.debug.print("\nf64：{d:.15}\n", .{float64});
    std.debug.print("f32：{d:.15}\n", .{float32});

    // 浮點到整數（向零截斷）
    const pi: f64 = 3.99999;
    const pi_int: i32 = @intFromFloat(pi);  // 變成 3

    std.debug.print("\n浮點 {d:.5} 到整數：{d}\n", .{ pi, pi_int });

    // 整數到浮點
    const count: i32 = 42;
    const count_float: f64 = @floatFromInt(count);

    std.debug.print(" 整數 {d} 到浮點：{d}\n", .{ count, count_float });
}
```

要點

1. 揀啱大小 - 當 i8 夠用時唔好用 i64
2. 有符號 **vs** 無符號 - 用無符號做計數/索引，用有符號做算術
3. 浮點精度 - f64 更精確但用更多記憶體
4. 顯式轉型 - Zig 要求你喺收窄時顯式轉型
5. 編譯時類型 - 冇類型嘅數字字面量係 `comptime_int` 或 `comptime_figure`

第四章：運算子

運算子係對值執行操作嘅符號。Zig 提供一套完整嘅算術、比較、邏輯同位元運算子。

運算子類別

```
┌──────────────────────────────────────────────────────────────┐
│                          運算子類別                          │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   算術              比較              邏輯                   │
│   ──────────        ──────────        ────────               │
│   +   加法          ==   等於         and   邏輯與           │
│   -   減法          !=   不等於       or    邏輯或           │
│   *   乘法          <    小於         !     邏輯非           │
│   /   除法          >    大於                                 │
│   %   取模          <=   小於或等於                           │
│   -   取負          >=   大於或等於                           │
│                                                              │
│   位元              賦值              特殊                   │
│   ────────          ──────────        ────────               │
│   &   與            =    賦值         ++   (Zig 冇！)        │
│   |   或            +=   加賦值       --   (Zig 冇！)        │
│   ^   異或          -=   減賦值       ?:   (用 if-else)      │
│   ~   非            *=   乘賦值                              │
│   <<  左移          /=   除賦值                              │
│   >>  右移          %=   取模賦值                            │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

算術運算子

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 算術運算子 ===\n\n", .{});

    const a: i32 = 17;
    const b: i32 = 5;

    // 基本運算
```

```zig
    std.debug.print("{d} + {d} = {d}\n", .{ a, b, a + b });   // 22
    std.debug.print("{d} - {d} = {d}\n", .{ a, b, a - b });   // 12
    std.debug.print("{d} * {d} = {d}\n", .{ a, b, a * b });   // 85

    // Zig 入面嘅除法需要顯式函數
    std.debug.print("{d} / {d} = {d}(截斷)\n", .{ a, b, @divTrunc(a, b) });

    // 取模
    std.debug.print("{d} %% {d} = {d}\n", .{ a, b, @mod(a, b) });

    // 取負
    std.debug.print("-{d} = {d}\n", .{ a, -a });

    // ========================================
    // 溢出安全運算子
    // ========================================

    std.debug.print("\n=== 溢出處理 ===\n\n", .{});

    // 環繞運算子(溢出時環繞)
    var x: u8 = 250;
    x +%= 10;  // 環繞:250 + 10 = 260 -> 4 (260 % 256)
    std.debug.print("250 +%% 10 = {d}(環繞)\n", .{x});

    // 飽和運算子(喺最大/最小值鉗位)
    var y: u8 = 250;
    y +|= 10;  // 飽和:250 + 10 = 255(u8 最大值)
    std.debug.print("250 +| 10 = {d}(飽和)\n", .{y});
}
```

輸出:

```
=== 算術運算子 ===

17 + 5 = 22
17 - 5 = 12
17 * 5 = 85
17 / 5 = 3(截斷)
17 % 5 = 2
-17 = -17

=== 溢出處理 ===

250 +% 10 = 4(環繞)
250 +| 10 = 255(飽和)
```

比較運算子

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 比較運算子 ===\n\n", .{});
```

```zig
    const x: i32 = 10;
    const y: i32 = 20;
    const z: i32 = 10;

    // 等於
    std.debug.print("{d} == {d}：{}\n", .{ x, y, x == y });   // false
    std.debug.print("{d} == {d}：{}\n", .{ x, z, x == z });   // true
    std.debug.print("{d} != {d}：{}\n", .{ x, y, x != y });   // true

    // 關係
    std.debug.print("\n{d} <  {d}：{}\n", .{ x, y, x < y });  // true
    std.debug.print("{d} >  {d}：{}\n", .{ x, y, x > y });    // false
    std.debug.print("{d} <= {d}：{}\n", .{ x, z, x <= z });   // true
    std.debug.print("{d} >= {d}：{}\n", .{ x, y, x >= y });   // false

    // 比較結果係布林值
    const is_equal: bool = x == z;
    const is_greater: bool = x > y;

    std.debug.print("\n儲存嘅結果：is_equal={}，is_greater={}\n", .{ is_equal, is_greater });
}
```

邏輯運算子

```
┌──────────────────────────────────────────────────────────────────────┐
│                              邏輯運算子                              │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   AND (and)：兩個都必須係 true                                       │
│   ┌──────────────────────────────────────────────────────────┐       │
│   │  A      │  B      │  A and B                              │       │
│   ├─────────┼─────────┼───────────────────────────────────────┤       │
│   │ false   │ false   │   false                               │       │
│   │ false   │ true    │   false                               │       │
│   │ true    │ false   │   false                               │       │
│   │ true    │ true    │   true   ✓                            │       │
│   └──────────────────────────────────────────────────────────┘       │
│                                                                      │
│   OR (or)：至少一個必須係 true                                       │
│   ┌──────────────────────────────────────────────────────────┐       │
│   │  A      │  B      │  A or B                               │       │
│   ├─────────┼─────────┼───────────────────────────────────────┤       │
│   │ false   │ false   │   false                               │       │
│   │ false   │ true    │   true   ✓                            │       │
│   │ true    │ false   │   true   ✓                            │       │
│   │ true    │ true    │   true   ✓                            │       │
│   └──────────────────────────────────────────────────────────┘       │
│                                                                      │
│   NOT (!)：反轉值                                                    │
│   ┌──────────────────────────────────────────────────────────┐       │
│   │  A      │  !A                                           │       │
```

```
│    ├─────────┼──────────────────────────────────────┤          │
│    │ false   │  true                                │          │
│    │ true    │  false                               │          │
│    └─────────┴──────────────────────────────────────┘          │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

位元運算子

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 位元運算子 ===\n\n", .{});

    const a: u8 = 0b11001010;  // 202
    const b: u8 = 0b10101100;  // 172

    std.debug.print("a     = 0b{b:0>8} ({d})\n", .{ a, a });
    std.debug.print("b     = 0b{b:0>8} ({d})\n", .{ b, b });

    std.debug.print("\na & b = 0b{b:0>8} ({d}) - 與\n", .{ a & b, a & b });
    std.debug.print("a | b = 0b{b:0>8} ({d}) - 或\n", .{ a | b, a | b });
    std.debug.print("a ^ b = 0b{b:0>8} ({d}) - 異或\n", .{ a ^ b, a ^ b });
    std.debug.print("~a    = 0b{b:0>8} ({d}) - 非\n", .{ ~a, ~a });

    // 位元移位
    const x: u8 = 0b00001111;  // 15
    std.debug.print("\nx      = 0b{b:0>8} ({d})\n", .{ x, x });
    std.debug.print("x << 2 = 0b{b:0>8} ({d}) - 左移\n", .{ x << 2, x << 2 });
    std.debug.print("x >> 2 = 0b{b:0>8} ({d}) - 右移\n", .{ x >> 2, x >> 2 });
}
```

要點

1. 冇 ++ 或–運算子 - 用 += 1 同 –= 1 代替
2. 顯式溢出處理 - 用 +%, –%, *% 做環繞，+|, –|, *| 做飽和
3. 短路求值 - and 同 or 喺唔需要時唔會求值第二個操作數
4. 位元運算子 - 對於旗標、掩碼同低階程式設計好有用
5. 除法函數 - 用 @divTrunc, @divFloor, @mod, @rem 做顯式行為

第五章：陣列

陣列係固定大小、連續嘅相同類型元素集合。佢哋係 Zig 入面最基本嘅數據結構之一。

建立陣列

```zig
const std = @import("std");

pub fn main() void {
```

```zig
    std.debug.print("=== 建立陣列 ===\n\n", .{});

    // 方法 1：顯式類型同值
    const explicit: [5]i32 = [5]i32{ 10, 20, 30, 40, 50 };
    std.debug.print(" 顯式：{any}\n", .{explicit});

    // 方法 2：用 [_] 推斷長度
    const inferred = [_]i32{ 1, 2, 3, 4 };  // 編譯器計數：4 個元素
    std.debug.print(" 推斷：{any}\n", .{inferred});
    std.debug.print(" 長度：{d}\n", .{inferred.len});

    // 方法 3：用相同值填充
    const zeros = [_]i32{0} ** 5;    // [0, 0, 0, 0, 0]
    const ones = [_]i32{1} ** 3;     // [1, 1, 1]
    const magic = [_]i32{42} ** 4;   // [42, 42, 42, 42]

    std.debug.print(" 零：{any}\n", .{zeros});
    std.debug.print(" 一：{any}\n", .{ones});
    std.debug.print(" 魔術：{any}\n", .{magic});

    // 方法 4：重複模式
    const pattern = [_]i32{ 1, 2 } ** 3;  // [1, 2, 1, 2, 1, 2]
    std.debug.print(" 模式：{any}\n", .{pattern});
}
```

要點

1. 固定大小 - 陣列長度喺編譯時已知，唔可以改變
2. 零索引 - 第一個元素喺索引 0
3. 邊界檢查 - Zig 喺調試模式檢查陣列邊界
4. 連續記憶體 - 元素連續儲存
5. 用 **[_]** 推斷長度 - 編譯器幫你計數元素

第六章：切片

切片係對陣列或其他連續記憶體嘅視圖。唔似陣列，切片唔擁有數據——佢哋只係引用佢。

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 建立切片 ===\n\n", .{});

    const array = [_]i32{ 10, 20, 30, 40, 50, 60, 70 };

    // 切片整個陣列
    const all: []const i32 = &array;
    std.debug.print(" 整個陣列：{any}\n", .{all});

    // 用範圍 [start..end) 切片
    const middle = array[2..5];  // 索引 2, 3, 4 嘅元素
    std.debug.print("middle[2..5]：{any}\n", .{middle});
```

```
    // 開放式切片
    const from_start = array[0..3];   // 頭 3 個元素
    const to_end = array[4..];        // 由索引 4 到尾
    std.debug.print("array[0..3]：{any}\n", .{from_start});
    std.debug.print("array[4..]：{any}\n", .{to_end});
}
```

要點

1. 切片唔擁有數據 - 佢哋係對現有記憶體嘅視圖
2. 範圍語法 **[start..end]** - 結尾係唔包括嘅
3. 靈活嘅函數參數 - 用 []const T 接受任何大小
4. 運行時長度 - 唔似陣列，切片長度可以喺運行時計算

第七章：指標

指標儲存記憶體地址。佢哋對系統程式設計好重要，允許你引用數據而唔複製佢。

```
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 指標基礎 ===\n\n", .{});

    // 建立一個變數
    var value: i32 = 42;

    // 用 & 獲取指向佢嘅指標
    const ptr: *i32 = &value;

    // 用 .* 讀取指標
    std.debug.print("value = {d}\n", .{value});
    std.debug.print("ptr.* = {d}\n", .{ptr.*});

    // 通過指標修改
    ptr.* = 100;
    std.debug.print("\nptr.* = 100 之後：\n", .{});
    std.debug.print("value = {d}\n", .{value});
}
```

要點

1. **&** 獲取地址 - 用 &x 獲取指向 x 嘅指標
2. **.*** 解引用 - 用 ptr.* 訪問值
3. 指標啟用修改 - 傳指標嚟修改原始數據
4. **Const** 指標 - *const T 防止通過該指標修改
5. 預設冇 **null** - 普通指標唔可以係 null；用 ?*T 做可空

第八章：字串

喺 Zig 入面，字串只係位元組陣列（`[]const u8`）。冇特殊嘅字串類型——呢個令語言保持簡單，畀你完全控制。

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 字串基礎 ===\n\n", .{});

    const greeting: []const u8 = "Hello, Zig!";

    std.debug.print(" 字串：{s}\n", .{greeting});
    std.debug.print(" 長度：{d}\n", .{greeting.len});

    // 訪問個別位元組
    std.debug.print(" 第一個位元組：'{c}' (ASCII {d})\n", .{ greeting[0], greeting[0] });

    // 遍歷字元
    std.debug.print("\n字元：", .{});
    for (greeting) |char| {
        std.debug.print("{c} ", .{char});
    }
    std.debug.print("\n", .{});
}
```

要點

1. 字串係 **`[]const u8`** - 只係位元組切片，冇乜特別
2. 用 **`std.mem`** 做操作 - `eql`, `indexOf`, `startsWith` 等
3. **`++`** 連接只喺編譯時 - 運行時唔可以連接
4. 預設 **UTF-8** - Zig 字串係 UTF-8 編碼
5. **`{s}`** 格式說明符 - 用嚟印字串

第九章：結構

結構讓你將相關數據組合成自定義類型。

```zig
const std = @import("std");

const Point = struct {
    x: i32,
    y: i32,
};

const Circle = struct {
    x: f64,
    y: f64,
    radius: f64,

    // 方法 – 接受 self 作為參數
    pub fn area(self: Circle) f64 {
```

```
        return std.math.pi * self.radius * self.radius;
    }
};

pub fn main() void {
    std.debug.print("=== 建立結構 ===\n\n", .{});

    const p1 = Point{ .x = 10, .y = 20 };
    std.debug.print("p1：({d}, {d})\n", .{ p1.x, p1.y });

    const c1 = Circle{ .x = 0, .y = 0, .radius = 5 };
    std.debug.print(" 圓面積：{d:.2}\n", .{c1.area()});
}
```

要點

1. 命名字段 - 用 `.fieldname` 訪問
2. 預設值 - 字段可以有預設值
3. 方法 - 帶 `self` 參數嘅函數
4. 關聯函數 - 冇 `self` 嘅函數（似 static）

第十章：枚舉

枚舉定義有固定命名值集合嘅類型。

```
const std = @import("std");

const Direction = enum {
    north,
    south,
    east,
    west,

    pub fn opposite(self: Direction) Direction {
        return switch (self) {
            .north => .south,
            .south => .north,
            .east => .west,
            .west => .east,
        };
    }
};

pub fn main() void {
    std.debug.print("=== 枚舉基礎 ===\n\n", .{});

    const dir: Direction = .north;
    std.debug.print(" 方向：{}\n", .{dir});
    std.debug.print(" 相反：{}\n", .{dir.opposite()});
}
```

要點

1. 窮盡式 **switch** - 必須處理每個枚舉值
2. 枚舉上概方法 - 枚舉可以有自己概方法
3. 短語法 - 用 `.value` 代替 `EnumType.value`

第十一章：聯合

聯合可以持有幾種類型之一，但一次只能持有一種。

```
const std = @import("std");

const Value = union(enum) {
    int: i64,
    float: f64,
    text: []const u8,
    none: void,

    pub fn describe(self: Value) void {
        switch (self) {
            .int => |n| std.debug.print(" 整數：{d}\n", .{n}),
            .float => |f| std.debug.print(" 浮點：{d:.2}\n", .{f}),
            .text => |s| std.debug.print(" 文字：{s}\n", .{s}),
            .none => std.debug.print(" 無\n", .{}),
        }
    }
};

pub fn main() void {
    std.debug.print("=== 標籤聯合 ===\n\n", .{});

    const values = [_]Value{
        .{ .int = 42 },
        .{ .float = 3.14159 },
        .{ .text = "Hello" },
        .none,
    };

    for (values) |v| {
        v.describe();
    }
}
```

第十二章：控制流程

Zig 提供標準控制流程：`if`、`switch`、`while` 同 `for`。每個都可以用作表達式。

```
const std = @import("std");

pub fn main() void {
```

```
    // If 表達式
    const x: i32 = 42;
    const sign = if (x > 0) " 正數" else if (x < 0) " 負數" else " 零";
    std.debug.print("{d} 係 {s}\n", .{ x, sign });

    // Switch 表達式
    const day: u8 = 3;
    const name = switch (day) {
        1 => " 星期一",
        2 => " 星期二",
        3 => " 星期三",
        4 => " 星期四",
        5 => " 星期五",
        6, 7 => " 週末",
        else => " 無效",
    };
    std.debug.print(" 日 {d} 係 {s}\n", .{ day, name });

    // For 循環
    const numbers = [_]i32{ 10, 20, 30 };
    std.debug.print(" 數字：", .{});
    for (numbers) |n| {
        std.debug.print("{d} ", .{n});
    }
    std.debug.print("\n", .{});
}
```

第十三章：函數

函數係可重用嘅程式碼塊。

```
const std = @import("std");

fn add(a: i32, b: i32) i32 {
    return a + b;
}

fn greet(name: []const u8) void {
    std.debug.print(" 你好，{s}！\n", .{name});
}

fn factorial(n: u64) u64 {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

pub fn main() void {
    std.debug.print("=== 函數 ===\n\n", .{});

    std.debug.print("5 + 3 = {d}\n", .{add(5, 3)});
    greet("Zig");
```

```
    std.debug.print("5! = {d}\n", .{factorial(5)});
}
```

第十四章：錯誤處理

Zig 有內置嘅錯誤處理同錯誤聯合。錯誤係值，唔係異常。

```
const std = @import("std");

const MathError = error{
    DivisionByZero,
    NegativeNumber,
};

fn divide(a: i32, b: i32) MathError!i32 {
    if (b == 0) return error.DivisionByZero;
    return @divTrunc(a, b);
}

pub fn main() void {
    std.debug.print("=== 錯誤處理 ===\n\n", .{});

    // 用 if 檢查
    if (divide(10, 2)) |result| {
        std.debug.print("10 / 2 = {d}\n", .{result});
    } else |err| {
        std.debug.print(" 錯誤：{}\n", .{err});
    }

    // catch 帶預設值
    const safe = divide(10, 0) catch 0;
    std.debug.print("10 / 0（帶預設）: {d}\n", .{safe});
}
```

要點

1. 錯誤係值 - 唔係異常，係普通返回值
2. **try** 傳播 - 如果發生錯誤立即返回
3. **catch** 處理 - 提供預設值或本地處理
4. 自定義錯誤集 - 定義你自己嘅錯誤類型

第十五章：可選值

可選值代表可能唔存在嘅值。佢哋比空指標更安全。

```
const std = @import("std");

fn find(haystack: []const u8, needle: u8) ?usize {
    for (haystack, 0..) |byte, index| {
```

```
        if (byte == needle) return index;
    }
    return null;
}

pub fn main() void {
    std.debug.print("=== 可選值 ===\n\n", .{});

    const text = "Hello, World!";
    if (find(text, 'W')) |index| {
        std.debug.print("'W' 喺索引 {d} 搵到\n", .{index});
    }

    if (find(text, 'Z')) |index| {
        std.debug.print("'Z' 喺索引 {d} 搵到\n", .{index});
    } else {
        std.debug.print("'Z' 搵唔到\n", .{});
    }

    // orelse - 提供預設值
    const val: ?i32 = null;
    std.debug.print("val orelse -1：{d}\n", .{val orelse -1});
}
```

要點

1. **?T** = 可能係 **T** - 值或 null
2. **if (opt) |val|** - 安全解包帶載荷捕獲
3. **orelse** - 如果係 null 提供預設值
4. **.?** 運算子 - 解包（如果係 null 會 panic）

第十六章：Comptime

Comptime 喺編譯時執行程式碼。呢個啟用強大嘅元程式設計而唔需要宏。

```
const std = @import("std");

const KILOBYTE = 1024;
const MEGABYTE = KILOBYTE * 1024;
const GIGABYTE = MEGABYTE * 1024;

fn factorial(n: u64) u64 {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

const FACT_10 = factorial(10);

pub fn main() void {
    std.debug.print("=== COMPTIME ===\n\n", .{});
```

```
    std.debug.print("1 KB = {d} 位元組\n", .{KILOBYTE});
    std.debug.print("1 MB = {d} 位元組\n", .{MEGABYTE});
    std.debug.print("1 GB = {d} 位元組\n", .{GIGABYTE});

    std.debug.print("\n10! = {d}（喺編譯時計算！）\n", .{FACT_10});
}
```

第十七章：泛型

泛型讓你寫可以同任何類型一齊用嘅程式碼。

```
const std = @import("std");

fn max(comptime T: type, a: T, b: T) T {
    return if (a > b) a else b;
}

fn swap(comptime T: type, a: *T, b: *T) void {
    const temp = a.*;
    a.* = b.*;
    b.* = temp;
}

pub fn main() void {
    std.debug.print("=== 泛型 ===\n\n", .{});

    std.debug.print("max(i32, 5, 10) = {d}\n", .{max(i32, 5, 10)});
    std.debug.print("max(f64, 3.14, 2.71) = {d}\n", .{max(f64, 3.14, 2.71)});

    var x: i32 = 100;
    var y: i32 = 200;
    std.debug.print("\n交換前：x={d}，y={d}\n", .{ x, y });
    swap(i32, &x, &y);
    std.debug.print(" 交換後：x={d}，y={d}\n", .{ x, y });
}
```

第十八章：記憶體管理

Zig 畀你對記憶體分配嘅顯式控制。

```
const std = @import("std");

pub fn main() !void {
    std.debug.print("=== 記憶體管理 ===\n\n", .{});

    // 堆疊記憶體（自動）
    var stack_var: i32 = 42;
    std.debug.print(" 堆疊變數：{d}\n", .{stack_var});
```

```zig
    // 堆記憶體（手動）
    var gpa = std.heap.GeneralPurposeAllocator(.{}){};
    defer _ = gpa.deinit();
    const allocator = gpa.allocator();

    // 分配單個值
    const ptr = try allocator.create(i32);
    defer allocator.destroy(ptr);
    ptr.* = 100;
    std.debug.print(" 堆值：{d}\n", .{ptr.*});

    // 分配切片
    const slice = try allocator.alloc(i32, 5);
    defer allocator.free(slice);
    for (slice, 0..) |*item, i| {
        item.* = @as(i32, @intCast((i + 1) * 10));
    }
    std.debug.print(" 堆切片：{any}\n", .{slice});
}
```

第十九章：測試

Zig 有內置測試支持。

```zig
const std = @import("std");
const expect = std.testing.expect;
const expectEqual = std.testing.expectEqual;

fn add(a: i32, b: i32) i32 {
    return a + b;
}

test " 基本加法" {
    try expectEqual(@as(i32, 5), add(2, 3));
    try expectEqual(@as(i32, 0), add(-5, 5));
}

test "expect 例子" {
    try expect(add(1, 1) == 2);
    try expect(10 > 5);
}

pub fn main() void {
    std.debug.print(" 用 zig test filename.zig 運行測試\n", .{});
}
```

用以下命令運行：`zig test filename.zig`

第二十章：C 互操作

Zig 可以直接調用 C 程式碼。

```zig
const std = @import("std");
const c = @cImport({
    @cInclude("math.h");
});

pub fn main() void {
    std.debug.print("=== C 互操作 ===\n\n", .{});

    const x: c.double = 2.0;

    std.debug.print("sqrt({d}) = {d}\n", .{ x, c.sqrt(x) });
    std.debug.print("pow({d}, 3) = {d}\n", .{ x, c.pow(x, 3.0) });
}
```

附錄 A：Zig 速查表

```
┌──────────────────────────────────────────────────────────────┐
│                          ZIG 速查表                          │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  變數                                                        │
│  ────────────────────────────────────────────────────        │
│  const x: i32 = 42;          // 不可變                       │
│  var y: i32 = 0;             // 可變                         │
│  var z: i32 = undefined;     // 未初始化                     │
│                                                              │
│  類型                                                        │
│  ────────────────────────────────────────────────────        │
│  i8, i16, i32, i64, i128     // 有符號整數                   │
│  u8, u16, u32, u64, u128     // 無符號整數                   │
│  f16, f32, f64, f128         // 浮點數                       │
│  bool                        // 布林值                       │
│  []T                         // 切片                         │
│  [N]T                        // 陣列                         │
│  *T                          // 指標                         │
│  ?T                          // 可選值                       │
│  !T 或 E!T                   // 錯誤聯合                     │
│                                                              │
│  控制流程                                                    │
│  ────────────────────────────────────────────────────        │
│  if (cond) { } else { }                                      │
│  switch (x) { val => {}, else => {} }                        │
│  while (cond) : (cont) { }                                   │
│  for (slice) |item| { }                                      │
│  for (slice, 0..) |item, i| { }                              │
```

```
函數
fn name(param: T) ReturnType { }
fn name(param: T) !ReturnType { }  // 可能出錯

錯誤處理
try expression           // 傳播錯誤
catch |err| { }          // 處理錯誤
orelse default           // 解包可選值
if (opt) |val| { }       // 解包可選值

記憶體
allocator.create(T)      // 分配一個
allocator.alloc(T, n)    // 分配多個
allocator.destroy(ptr)   // 釋放一個
allocator.free(slice)    // 釋放多個
defer cleanup();         // 喺作用域退出時執行

常用內建函數
@import("name")          // 導入模組
@sizeOf(T)               // 類型大小
@typeInfo(T)             // 類型信息
@intCast(val)            // 整數轉換
@floatCast(val)          // 浮點轉換
```

關於作者

JW Lee 係一個熱衷於系統程式設計嘅軟件開發者，致力於令複雜嘅話題對初學者易於理解。呢本書代表佢努力創建最全面但易於理解嘅 Zig 程式語言入門。

簡易 *Zig - Zig* 程式語言嘅完整初學者指南

版權所有 *2024 JW Lee*

呢本書入面嘅所有例子都係喺 *MIT* 許可證下提供。